

ARDUINO PHYSICAL AI CHALLENGE INDIA 2026

Project Report

Organised by Robu.in × Arduino · Submit as PDF · Max 10 MB

Project Title	HapticTalk — a bidirectional sign-language and haptic communication device
Team Name	Wallie
Registration / Team ID	APC-2026-TS-30175
Contest Track	Interactive AI
Institution & City	Shiv Nadar University, Delhi NCR

Team Members

Name	Email	Role
Sumedh Thampi	sumedhthampi@gmail.com	hardware, firmware, ML and software

1. Project Overview

Problem Statement

India has tens of millions of deaf and hard-of-hearing people and only a few hundred certified sign language interpreters. Without an interpreter, a deaf person cannot walk into a hospital and describe their symptoms, ask a stranger for help, or hold an ordinary conversation with someone who does not sign.

Writing notes is the usual workaround, but it is slow and requires both people to stop and type — which fails outright in an emergency.

For a person who is both deaf and blind the problem is harder still. They can sign a message outward, but speech and text both fail on the way back. Touch is the only remaining channel, and there is no affordable wearable device that uses it well.

HapticTalk addresses both directions with a single device.

How Your Project Works

HapticTalk translates in two directions. A camera watches the user sign. MediaPipe extracts 21 landmarks for each hand, and thirty consecutive frames — roughly two seconds — are packed into a 3,780-value window and streamed to the Arduino UNO Q over a socket.

On the board, a neural network trained in Edge Impulse classifies that window into one of 23 Indian Sign Language signs in 4–8 ms. Acceptance guards reject transitional noise, a pattern matcher assembles recognised signs into natural English, and the resulting sentence is spoken aloud using pre-rendered neural-voice audio.

The reply travels the other way. A spoken word is transcribed, matched against the vocabulary, and sent to a glove carrying five coin vibration motors — one per fingertip. Each word is

encoded as a distinct pulse pattern: the finger selects a category, the pulse sequence selects the word, and an envelope layered on top conveys intent (neutral, urgent, question, affirmation). The result is a loop that works when neither hearing nor sight is available: sign in, speech out; speech in, touch out. Everything runs offline on one board.

Why Arduino UNO Q?

The UNO Q is a dual-brain board — a Qualcomm QRB2210 running Linux alongside an STM32U585 microcontroller, linked by Arduino's Router Bridge RPC. That split is exactly what this project needs, and it is why the whole system fits on one device instead of two.

The Linux side runs the Edge Impulse classifier, the sentence grammar and the audio pipeline: work that needs a filesystem, a Python runtime and a Bluetooth audio stack. The microcontroller drives the vibration motors with the millisecond-accurate PWM timing the haptic encoding depends on, free of the scheduling jitter a general-purpose OS introduces.

Running inference on-device means no cloud round-trip, no API key and no internet dependency. For an assistive device used in a hospital corridor or on the street, that is not an optimisation — it is a requirement.

2. Components Used (BOM)

List every component. The Arduino UNO Q purchase proof must be uploaded separately with your submission.

Component	Qty
Arduino UNO Q (ABX00087)	1
Coin vibration motor, 3 V (2.5–4.0 V, 90 mA run, 120 mA start)	5
ULN2003 Darlington driver array	1
PAM8403 class-D amplifier module	1
Bluetooth speaker (JBL Go 3)	1
USB / laptop webcam	1
Anti-static ESD glove (chassis)	1
Breadboard power supply (isolated 5 V motor rail)	1
26 AWG silicone wire, tape, connectors	—

3. System Architecture & Circuit

Step-by-Step Workflow

1. A camera captures the signer. MediaPipe extracts 21 landmarks per hand — 126 float values per frame.
2. Thirty consecutive frames (~2 s at 15 fps) are buffered and streamed to the UNO Q over a TCP socket as CSV lines.

3. On the UNO Q's Linux side, the Edge Impulse model classifies the 3,780-value window into one of 23 signs in 4–11 ms.
4. Acceptance guards filter the result: the window must contain a hand in at least 80% of frames, the hand must have settled, and the same label must win several consecutive windows.
5. Accepted signs enter a buffer and a pattern matcher assembles them into a sentence (for example I + PAIN + DOCTOR becomes “I am in pain, please call a doctor.”).
6. The sentence is spoken through a pre-rendered neural-voice clip played to a Bluetooth speaker; if no clip exists the system falls back to live synthesis.
7. In parallel, every recognised sign is dispatched over Router Bridge RPC to the STM32U585, which plays the matching vibration pattern on the glove.
8. For the reply direction, a spoken word is transcribed, matched to the vocabulary and sent to the glove as a haptic pattern.

How the User Reads a Vibration

A vibration has to be decoded by a person, not a computer, so the encoding is constrained by what fingertips can actually resolve rather than by what is convenient to transmit. Two findings from testing shaped the whole scheme.

Encoding a word as several motors firing at once does not work. Vibration on adjacent fingertips merges into a single diffuse sensation, so “thumb plus middle plus ring” cannot reliably be told apart from “thumb plus index plus ring”. Counting pulses fails too — users lose track past three. Both approaches look efficient on paper and fail on skin.

What does work is one finger at a time, with meaning carried by pulse length. Short versus long is a near-binary perceptual judgement, and it yields more distinct patterns per unit time than counting does. Decoding then happens in three stages, resolved in this order:

WHERE — which finger vibrates gives the category — thumb is medical and urgent, index is needs, middle is identity, ring is social, pinky is responses. This is the most important decision in the scheme, because the category arrives before the word does: by the time the user is reading pulses, they are choosing between five options rather than twenty-three. Thumb and pinky, the two most perceptually distinct fingers, carry the vocabulary that matters most under stress.

WHAT — the pulse sequence on that finger selects the word. A short pulse is 100 ms, a long pulse 300 ms, with a 120 ms gap between them. One or two pulses give five words per finger, and the longest pattern completes in about 720 ms.

Code	Thumb — medical	Index — needs	Middle — identity	Ring — social	Pinky — responses
S	PAIN	WATER	I	TODAY	YES
L	HELP	FOOD	NAME	GOODMORNING	NO
S-S	DOCTOR	THIRSTY	DEAF	UNDERSTAND	PLEASE
S-L	HOSPITAL	—	BLIND	COMMUNICATION	THANKYOU
L-S	MEDICINE	—	AI	DEMONSTRATE	SORRY

23 words across five fingers, with seven slots free for expansion.

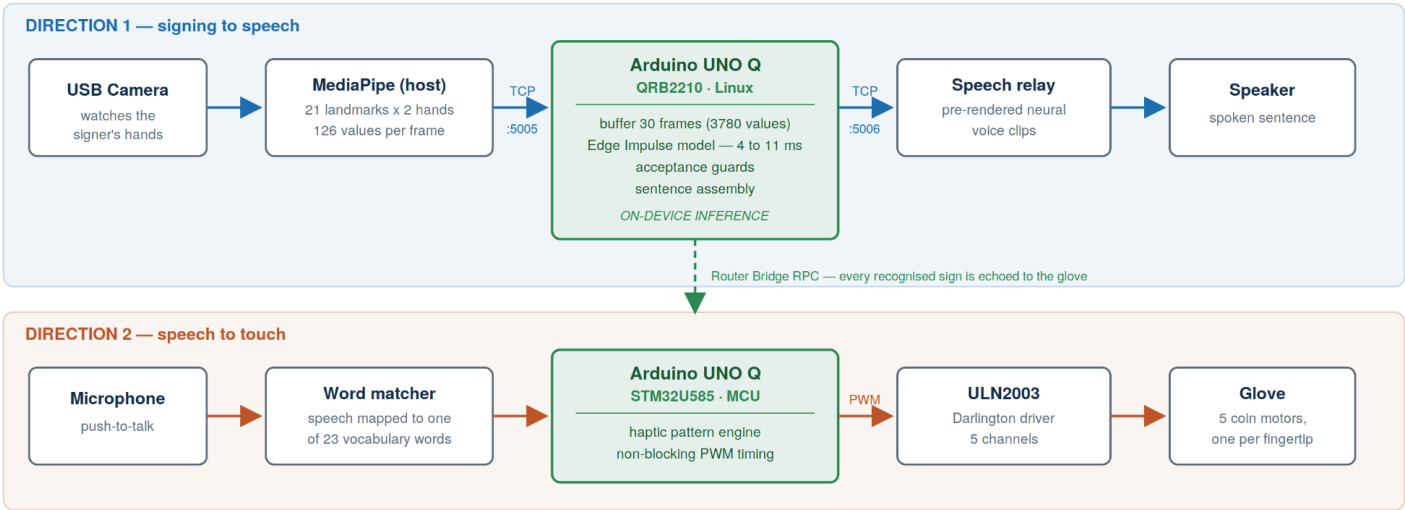
HOW — an envelope layered over any word conveys intent. Neutral plays at rated intensity. Urgent opens with an alert burst on thumb and pinky, then replays the word at higher intensity with timings compressed to 60 per cent. A question ramps intensity upward across the word and ends with a rising sweep from thumb to pinky. An affirmation adds a short double buzz on the same finger. The user therefore feels not only which word was said, but how it was meant.

One physical detail makes short pulses legible at all. Coin motors need roughly 2.3 V to break static friction and 30–50 ms to reach speed, so a 100 ms pulse at normal intensity would barely register on the skin. Every pulse therefore opens at full power for 25 ms before dropping to its target level, so the motor is already spinning by the time the pulse reaches the fingertip.

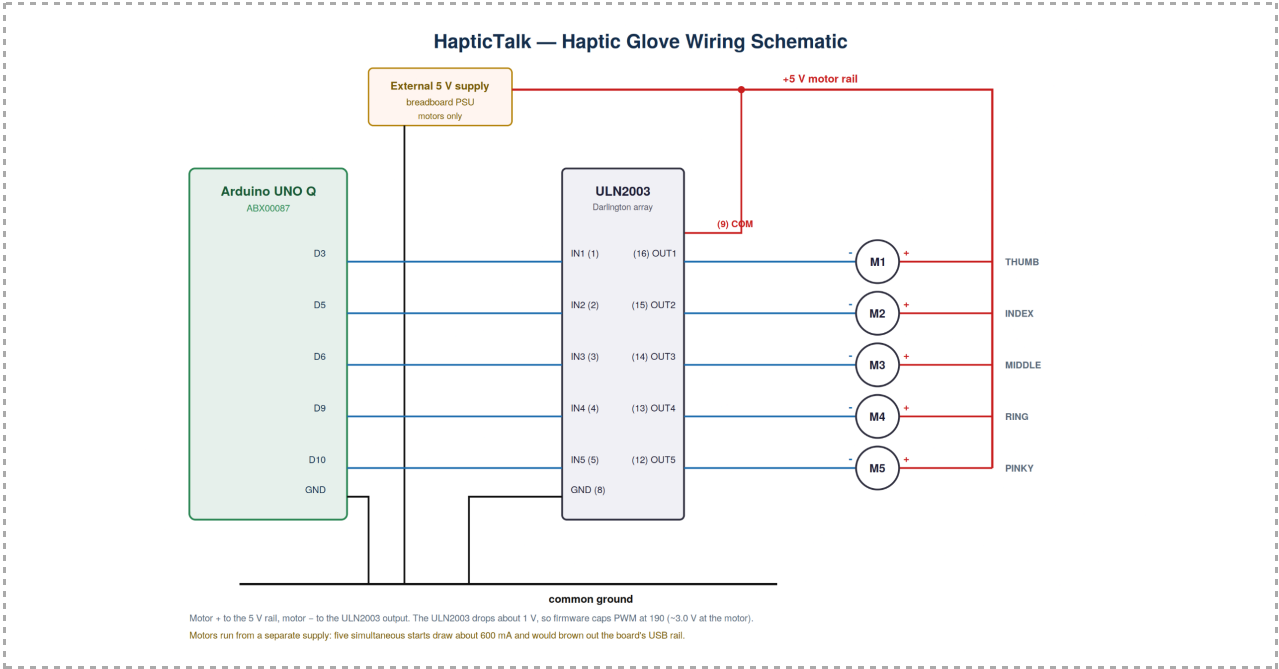
Block Diagram & Circuit Schematic

HapticTalk — System Block Diagram

All inference runs on the Arduino UNO Q. No cloud service in either direction.



Inference: 4–11 ms on-device · End-to-end loop: approximately 100–300 ms
Vocabulary: 23 Indian Sign Language signs



4. AI / ML Model Details

If your project does not use AI/ML, write “Not Applicable” — but note that Physical AI projects score higher on Innovation.

Model Used	Dense neural network — Flatten → Dense(20) → Dense(10) → 23-class softmax
Training Platform	Edge Impulse (deployed as a .eim runtime on the UNO Q)
Accuracy	80% on a held-out test set (~90% validation)
Dataset	23 ISL signs × 30 samples = 690 sequences; each sample is 30 frames × 126 landmark features (3,780 values)

Brief Description & Limitations

The model sees a two-second window of hand geometry, not an image. Each frame contributes 126 numbers — 21 landmarks × (x, y, z) for each hand — so the classifier learns the shape and motion of a sign rather than the appearance of a hand, which makes it robust to lighting and background.

Limitations. There is no “rest” class, so the network must return one of 23 labels for any input, including noise — and it does so with high confidence. Confidence thresholding proved useless as a filter; what works is requiring the same label across several consecutive overlapping windows, since genuine signs hold steady while noise flickers between labels.

Accuracy also degrades in three known ways: several sign pairs are imperfectly separated (UNDERSTAND/I, AI/COMMUNICATION/TODAY, PAIN/WATER); per-sign error compounds across sentences, so an eight-sign sentence completes cleanly far less often than 80% would suggest; and the entire dataset was recorded by a single signer, so generalisation across users is untested.

5. Code Structure

mediapipe_bridge.py (host laptop) — captures the webcam, extracts landmarks, and streams one CSV line per frame to the board. Also forwards haptic commands received from listen.py.

main.py (UNO Q, Linux) — landmark_server() accepts frames and buffers them; classify_window() runs inference and applies the acceptance guards; process_sign() maintains the sign buffer and matches phrase patterns; say() forwards finished sentences to the speech relay; play_haptic() dispatches the sign to the microcontroller over Router Bridge.

tts_relay.py (UNO Q, host side) — owns the audio path. Receives sentence text, looks it up in a manifest of pre-rendered clips, and plays the matching file; falls back to live speech synthesis when no clip exists.

haptic_engine.ino (STM32U585) — setup() initialises the five motor pins; loop() advances a non-blocking pattern state machine so playback never blocks the RPC link. A serial command interface allows patterns to be tested standalone.

record_signs.py / upload_to_ei.py — dataset capture and upload to Edge Impulse.

generate_audio.py renders the speech clips. listen.py provides push-to-talk speech-to-haptic input.

GitHub Repository	https://github.com/xsumedh-st/HapticTalk
Demo Video Link	https://drive.google.com/file/d/1h7RL--lDdUI9WiNI9D2UtEAgHeU0DWqu/view?usp=sharing

6. Testing & Results

Model accuracy was measured by Edge Impulse on a held-out test set the model never saw during training: 80% across 23 classes. Validation accuracy was around 90%; the gap is itself informative, and points to needing more samples per class rather than a different architecture.

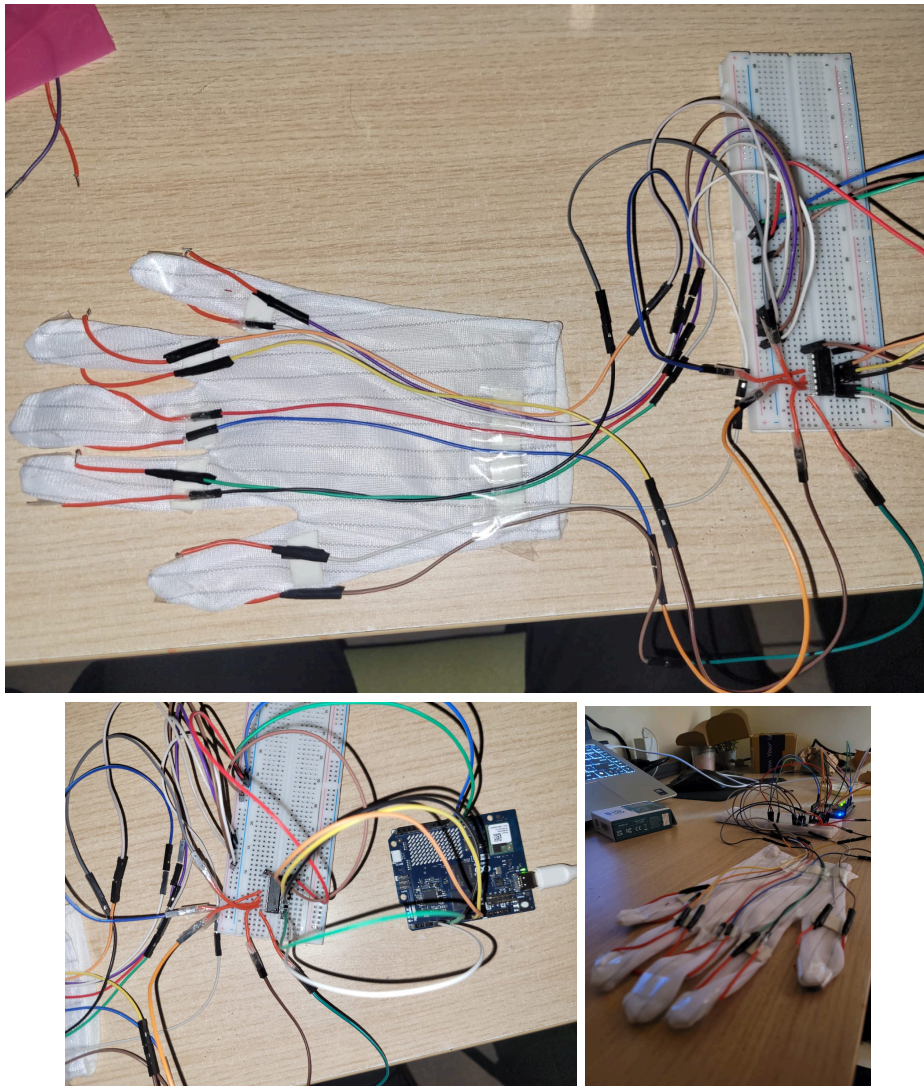
Latency. On-device inference measured 4–11 ms across several hundred classifications, logged live from the board. End-to-end latency — camera capture, landmark extraction, network transport, classification and audio playback — is roughly 100–300 ms.

Rejection tuning. Guards were tuned empirically against logged output. Before them, incidental hand movement produced false detections at up to 1.00 confidence, which established that confidence alone was not a usable filter. Requiring multi-window agreement removed almost all of it.

Haptics. Each motor was verified in place after mounting, and short-versus-long pulse discrimination was checked with a blind trial series on the assembled glove. Simultaneous-start current draw was measured to confirm the motor rail does not brown out; the urgent alert burst is deliberately limited to two motors for this reason.

Edge cases. Sentence assembly matches the tail of the sign buffer, so a spurious sign landing between two valid ones breaks the phrase and the system speaks only the last word. Signs that are the first word of a longer phrase wait for a timeout before speaking, which is correct behaviour but reads as latency.

Project Images (2–3)



7. Challenges, Learnings & Future Improvements

Challenges Faced

Audio from inside a container. The application runs in a Docker container with no sudo and no route to the host's audio session, so speech could not be produced where it was decided. The

fix was a small relay process on the host that owns the audio path and receives sentence text over a socket.

A classifier with no null class. The model must return one of 23 labels for any input, so it labels noise confidently. This was solved outside the model, with hand-presence, stillness and multi-window agreement guards.

Train/inference mismatch. Training data was captured at home before moving back to college. As the model was trained on 2-D values of hand position, moving to a different location and performing signs at a different height, made it harder to classify signs.

Motor physics. Coin ERM motors need about 2.3 V to break static friction and 30–50 ms to spin up, so short pulses were barely perceptible. Every pulse now opens at full PWM for 25 ms before dropping to its target level.

Perceptual limits. Encoding words as simultaneous multi-finger vibration failed in testing — adjacent fingertip sensations merge. The scheme was redesigned around one finger at a time, distinguished by pulse length.

Mechanical fragility. Motor leads pulled free at the solder joints when the glove was removed, which required strain relief and anchoring the harness to the fabric.

What You Learned & What's Next

What was built: a self-contained device that recognises 23 Indian Sign Language signs on-device in under 11 ms, speaks assembled sentences in a natural voice, and returns messages as learnable vibration patterns on a glove — with no cloud service anywhere in the loop.

What I learned: that most of the engineering in a physical AI project is not the model. The classifier took an afternoon; the difficult work was rejecting noise from a network with no null class, designing a haptic code that human fingertips can actually resolve, and making an embedded audio path work across a container boundary. I also learned to distrust confidence scores — measuring behaviour across time proved far more reliable than trusting a single number.

Next: (1) port MediaPipe to the QRB2210 so landmark extraction also runs on the board, making the device fully self-contained; (2) expand the dataset with multiple signers and add a rest class, which should fix both generalisation and false positives; (3) add a second glove for two-handed signs, battery power and a wireless link between glove and board.

8. Declaration

We confirm this is our original, unpublished work. The Arduino® UNO™ Q is the primary board in this project. All team members have reviewed and agree to this report.

Date	23-08-2026
------	------------